

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**DETAILED DESIGN SPECIFICATION
CSE 4317: SENIOR DESIGN II
SPRING 2026**



**THE KIOSK GUYS
SMART KIOSK**

**MIGUEL CERVANTES
PRAKASH SAPKOTA
EARL SANCHEZ
THIEN NGUYEN
IVAN KO**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	1.28.2026	MC	document creation
1.0	2.13.2026	MC, IK, ES	document first draft

CONTENTS

1	Introduction	6
2	System Overview	6
3	Hardware Layer Subsystems	7
3.1	Hardware Layer	7
3.2	Hardware Layer Operating System	7
3.3	Hardware Layer Software Dependencies	8
3.4	Subsystem 1: Kiosk Computer	8
3.5	Subsystem 2: Touchscreen Monitor	9
3.6	Subsystem 3: Card Swipe Reader	10
3.7	Subsystem 4: Wi-Fi Connection	11
4	Front-End Layer Subsystems	13
4.1	Front-End Layer Hardware	13
4.2	Front-End Layer Operating System	13
4.3	Front-End Layer Software Dependencies	13
4.4	Subsystem 1: Login/Registration User Interface	13
4.5	Subsystem 2: Supply Inventory User Interface	14
4.6	Subsystem 3: Interactive Map User Interface	15
4.7	Subsystem 4: Admin Dashboard User Interface	16
4.8	Subsystem 5: Room Reservation User Interface	17
4.9	Subsystem 6: Announcements Board User Interface	18
4.10	Subsystem 7: API Interactions	19
5	Back-End Layer Subsystems	21
5.1	Back-End Layer Hardware	21
5.2	Back-End Layer Operating System	21
5.3	Back-End Layer Software Dependencies	22
5.4	Subsystem 1: Django	22
5.5	Subsystem 2: Login Manager	23
5.6	Subsystem 3: Supply Order Manager	24
5.7	Subsystem 4: Automated Email System	25
5.8	Subsystem 5: Room Reservation Manager	26
5.9	Subsystem 6: Announcement Manager	27
5.10	Subsystem 7: Admin's Portal Manager	28
5.11	Subsystem 8: Website Hosting Service	29
6	Database Layer Subsystems	30
6.1	Database Layer Hardware	30
6.2	Database Layer Operating System	30
6.3	Database Layer Software Dependencies	30
6.4	Subsystem 1: TiDB	30
6.5	Subsystem 2: Django ORM	31
6.6	Subsystem 3: Database Queries and Operations	32

LIST OF FIGURES

1	System architecture	7
2	Hardware Layer	8
3	Kiosk Computer	8
4	Touchscreen Monitor	9
5	Card Swipe Reader	10
6	Wi-Fi Connection	11
7	Front-End Layer	13
8	Login/Registration UI	14
9	Supply Inventory UI	15
10	Interactive Map UI	16
11	Admin Dashboard UI	17
12	Room Reservation UI	17
13	Announcement Board UI	18
14	API Interactions	19
15	Back-End Layer	21
16	Django	22
17	Login Manager	23
18	Supply Order Manager	24
19	Automated Email System with API	25
20	Room Reservation Manager	26
21	Announcement Manager	27
22	Admin's Portal Manager	28
23	Website Hosting Service	29
24	TiDB	30
25	Django ORM	32
26	Database Queries and Operations	33

LIST OF TABLES

1 INTRODUCTION

The Smart Kiosk is a physical, interactive kiosk system designed for UTA's Engineering Research South A (ERSA) and Engineering and Research Building (ERB) to improve accessibility and convenience for staff, students, and visitors by centralizing common building-related services. The system consists of standalone kiosks placed in the main lobby of ERSA and at the main entrances of the ERB, with functionality tailored to each building's purpose. The ERB kiosks provide limited features such as an interactive map and announcement board for students and visitors, whereas the ERSA kiosk includes additional capabilities such as user login with card swipe input option, supply ordering, and room reservation to support faculty and research staff. The application is also hosted as a web-based platform for remote access, but with intentionally restricted functionality to encourage on-site kiosk use. Overall, the deployment includes one fully featured kiosk at ERSA, two limited-function kiosks at the ERB, and a remote website with constrained capabilities.

This document builds upon the system requirements and architectural decisions described in the Software Requirements Specification (SRS), Architectural Design Specification (ADS), and the Project Charter. The Project Charter defines the overall objectives, scope, stakeholders, and constraints of the Smart Kiosk project, establishing the foundation for its development. The SRS outlines the functional goals of the system, including room reservations, supply requests, building navigation, and content display, as well as the needs of different user types. The ADS details the system's overall structure, including the selected hardware platform and software stack used to support these requirements. Together, these documents provide the background, scope, and rationale that inform the detailed design presented in the following sections.

2 SYSTEM OVERVIEW

Figure 1 is a high level architectural overview of the Smart Kiosk system and illustrates how the front end, back end, database, and hardware layers interact to support both kiosk based and remote functionality. The front end layer contains multiple user interfaces such as login and registration, supply inventory, room reservation, announcements, and interactive maps, all of which communicate through a centralized API interaction component. This layer is responsible for handling user input and basic validation before forwarding requests to the back end.

The back end layer serves as the core of the system and includes system services as well as supporting infrastructure components such as the Email API and Website Hosting Service. It coordinates function logic through dedicated managers for login, room reservations, supply ordering, announcements, and administrative tasks, while also integrating external services like email delivery and remote web access. These services communicate with the database layer through the Django ORM, which manages database queries and operations on the TiDB database. The hardware layer connects the physical components of the kiosk, including the touchscreen display, kiosk computer, Wi Fi connection, and optional card swipe reader, to the software system. Together, the layers form a modular and scalable architecture that supports full functionality at the ERSA kiosk, limited functionality at the ERB kiosks, and restricted remote access via the hosted website.

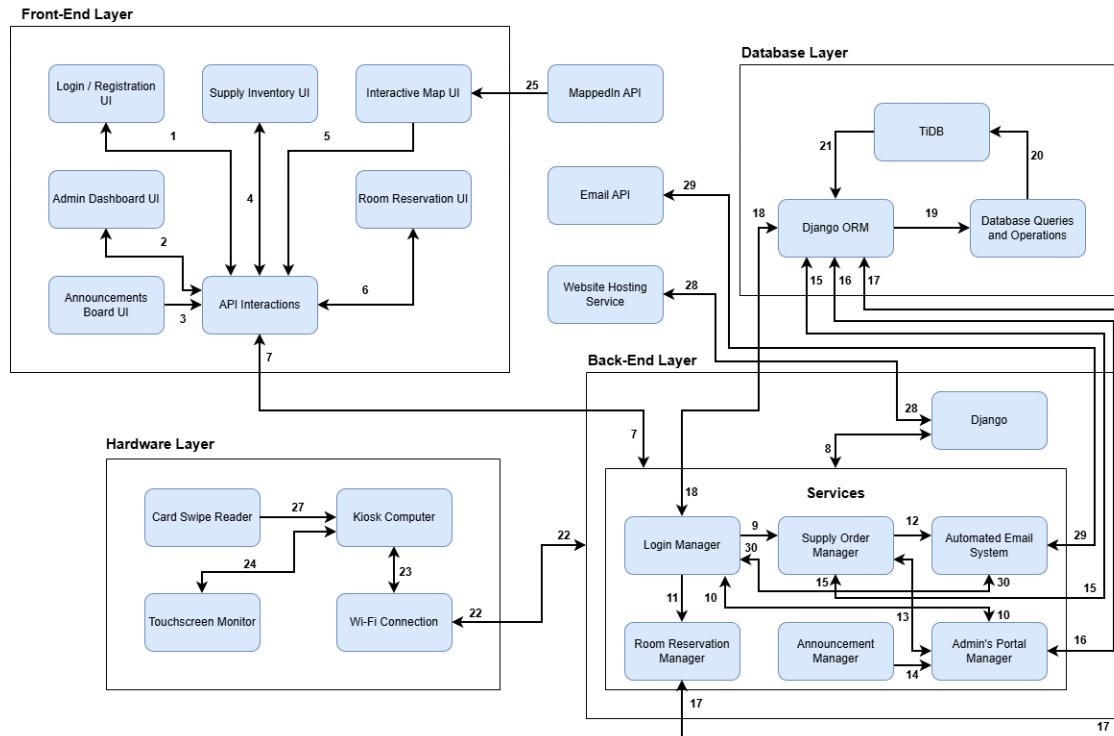


Figure 1: System architecture

3 HARDWARE LAYER SUBSYSTEMS

The Hardware Layer for the Smart Kiosk is responsible for displaying the application UI and capturing user touch input. Its subsystems include a central computing unit, a touchscreen monitor, a card swipe reader, and a stable Wi-Fi connection. Each kiosk includes all listed subsystems, except for those deployed in the Engineering and Research Building (ERB), which exclude the card swipe reader since it is not required to meet system requirements. This section provides detailed specifications for each listed subsystem.

3.1 HARDWARE LAYER

- Central computing unit in the form of a mini PC with sufficient processing power and memory to support all kiosk functionalities.
- Touchscreen monitor with multi-touch capabilities and a high-resolution, vibrant color display.
- Card swipe reader for reading user ID cards via a USB connection to the mini PC.
- Internet connectivity supporting Wi-Fi 6 through the mini PC's built-in Wi-Fi module, with optional wired Ethernet support.

3.2 HARDWARE LAYER OPERATING SYSTEM

All subsystems in this layer interface with the mini PC, which runs a compatible operating system such as Windows or Linux. The operating system is responsible for managing hardware resources and providing the runtime environment required for the kiosk application.

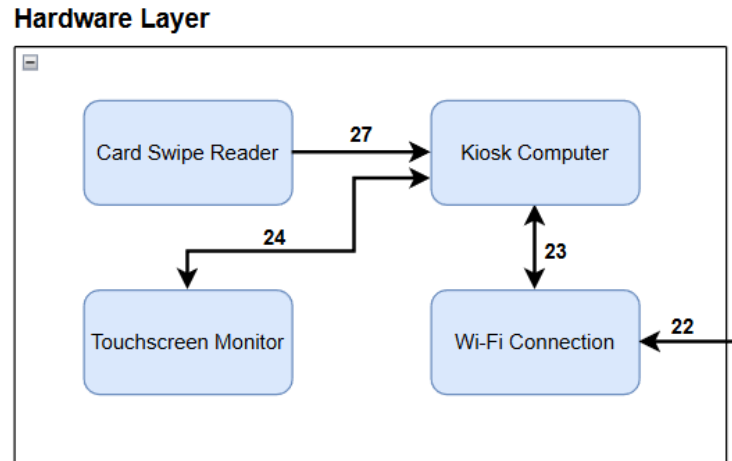


Figure 2: Hardware Layer

3.3 HARDWARE LAYER SOFTWARE DEPENDENCIES

The operating system must be compatible with the software stack used by the kiosk application, including a React front-end and a Django back-end, with further dependency specifications described later in the document.

3.4 SUBSYSTEM 1: KIOSK COMPUTER

The Kiosk Computer is a hardware subsystem in the form of a mini PC that serves as the central processing unit of the kiosk. It is responsible for receiving user input from the touchscreen monitor and card swipe reader, executing the kiosk's front-end and back-end applications, and managing communication with the database over a stable internet connection. Peripheral devices interface with the Kiosk Computer via USB connections, while network connectivity enables data synchronization and remote services.

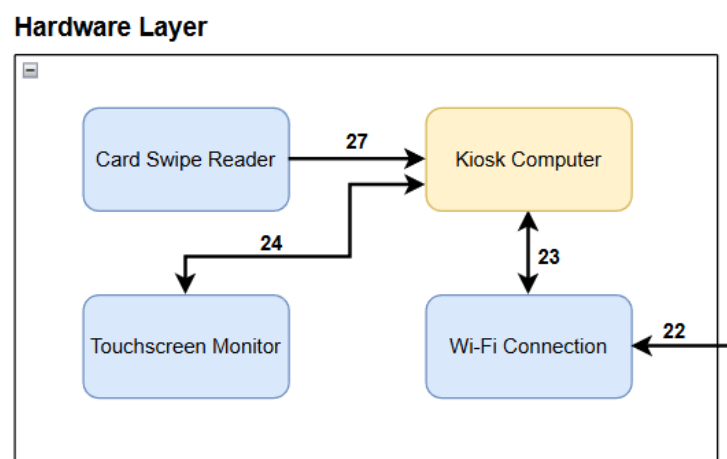


Figure 3: Kiosk Computer

3.4.1 KIOSK COMPUTER HARDWARE

Central computing unit in the form of a mini PC with sufficient processing power and memory to support all kiosk functionalities.

3.4.2 KIOSK COMPUTER OPERATING SYSTEM

The mini PC operates on a supported operating system such as Windows or Linux.

3.4.3 KIOSK COMPUTER SOFTWARE DEPENDENCIES

The operating system must be compatible with the software stack used by the kiosk application, including a React front-end and a Django back-end, with further dependency specifications described later in the document. Additionally, an on-screen keyboard feature must be available to enable touchscreen functionality.

3.4.4 KIOSK COMPUTER PROGRAMMING LANGUAGES

N/A. This subsystem represents a hardware component and does not directly implement or execute application-level code. Programming languages are specified in the software layers.

3.4.5 KIOSK COMPUTER DATA STRUCTURES

The Kiosk Computer facilitates data transfer between hardware peripherals, the front-end interface, the back-end services, and the database. At this layer, data structures are primarily concerned with the transport of input and output data rather than application-specific representations. Detailed data structures, formats, and protocols used by the front-end, back-end, and database are described in later sections of this document.

3.4.6 KIOSK COMPUTER DATA PROCESSING

N/A. The Kiosk Computer functions primarily as a hardware execution platform and does not implement subsystem-specific data processing algorithms. All application-level data processing and algorithmic logic are handled within the software layer and are described in later sections.

3.5 SUBSYSTEM 2: TOUCHSCREEN MONITOR

The touchscreen monitor serves as the primary user interface of the kiosk by displaying application pages and capturing user input through multi-touch functionality.

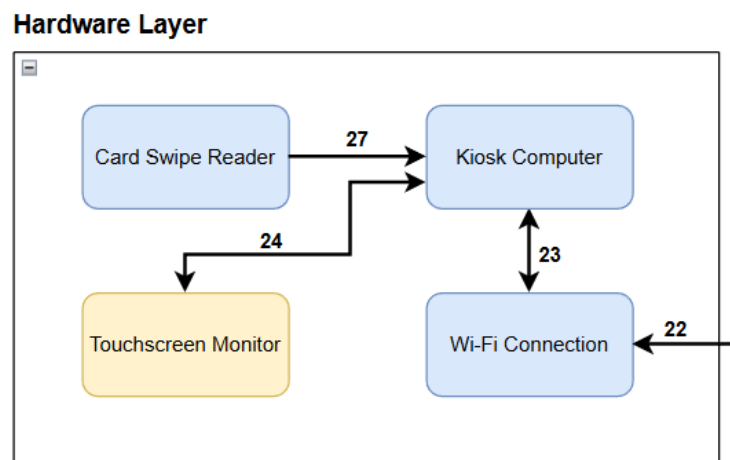


Figure 4: Touchscreen Monitor

3.5.1 TOUCHSCREEN MONITOR HARDWARE

Touchscreen monitor with multi-touch capabilities and a high-resolution, vibrant color display.

3.5.2 TOUCHSCREEN MONITOR OPERATING SYSTEM

N/A. The touchscreen monitor is a hardware peripheral and does not run an operating system. Touch input and display output are handled by the kiosk computer's operating system.

3.5.3 TOUCHSCREEN MONITOR SOFTWARE DEPENDENCIES

N/A. The touchscreen monitor is a hardware peripheral and does not require application-level software dependencies.

3.5.4 TOUCHSCREEN MONITOR PROGRAMMING LANGUAGES

N/A. The touchscreen monitor is a hardware peripheral and does not require application-level programming languages.

3.5.5 TOUCHSCREEN MONITOR DATA STRUCTURES

Touch input is provided to the kiosk computer as standard HID touch events through a USB connection, and display output is provided as a video signal through HDMI. Any application-specific data structures are defined in the software layers.

3.5.6 TOUCHSCREEN MONITOR DATA PROCESSING

N/A. The touchscreen monitor does not perform application-level data processing or implement algorithms.

3.6 SUBSYSTEM 3: CARD SWIPE READER

The card swipe reader provides an alternative input method for user authentication by reading data encoded on the magnetic stripe of a UTA identification card. The reader interfaces with the kiosk computer via a USB connection and transmits card data for processing by the kiosk application.

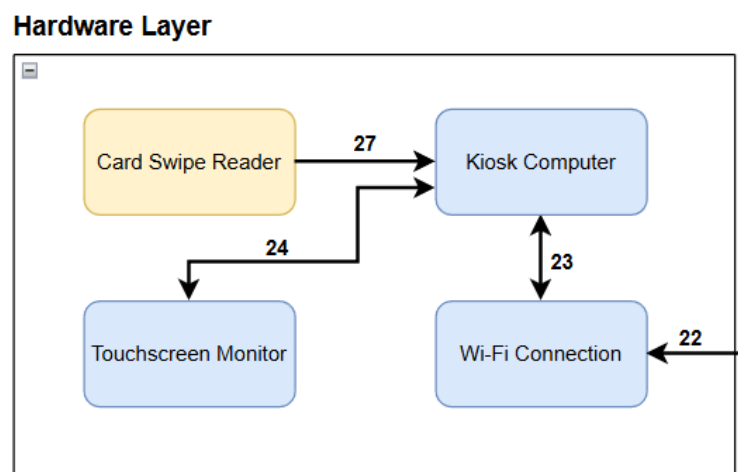


Figure 5: Card Swipe Reader

3.6.1 CARD SWIPE READER HARDWARE

- Card swipe reader for reading user ID cards via a USB connection to the mini PC.

3.6.2 CARD SWIPE READER OPERATING SYSTEM

N/A. The card swipe reader is a hardware peripheral and does not run an operating system. Data processing is handled by the kiosk computer's operating system.

3.6.3 CARD SWIPE READER SOFTWARE DEPENDENCIES

N/A. The card swipe reader is a hardware peripheral and does not require application-level software dependencies.

3.6.4 CARD SWIPE READER PROGRAMMING LANGUAGES

N/A. The card swipe reader is a hardware peripheral and does not require application-level programming languages.

3.6.5 CARD SWIPE READER DATA STRUCTURES

The card swipe reader produces structured magnetic stripe data upon a successful swipe. This data typically consists of encoded character strings representing card identification information, transmitted to the kiosk computer over a USB interface.

3.6.6 CARD SWIPE READER DATA PROCESSING

N/A. The card swipe reader does not perform application-level data processing or implement algorithms.

3.7 SUBSYSTEM 4: WI-FI CONNECTION

The kiosk requires a stable internet connection to access the web-based application and its associated database services. Network connectivity is provided through the mini PC's integrated Wi-Fi module, with optional support for a wired Ethernet connection. The system supports Wi-Fi 6 to ensure reliable and high-performance wireless communication.

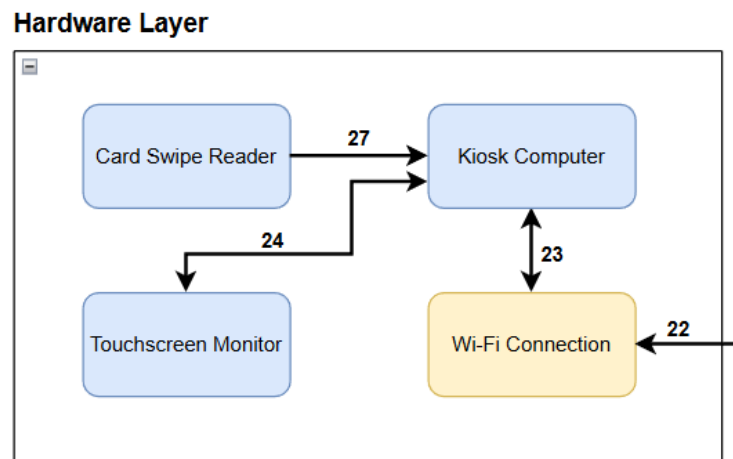


Figure 6: Wi-Fi Connection

3.7.1 WI-FI CONNECTION HARDWARE

- Internet connectivity supporting Wi-Fi 6 through the mini PC's built-in Wi-Fi module, with optional wired Ethernet support.

3.7.2 Wi-Fi CONNECTION OPERATING SYSTEM

N/A. The Wi-Fi connection is a networking capability provided by the mini PC's hardware and managed by the kiosk computer's operating system.

3.7.3 Wi-Fi CONNECTION SOFTWARE DEPENDENCIES

N/A. The Wi-Fi connection is a networking capability provided by the mini PC and does not require software dependencies.

3.7.4 Wi-Fi CONNECTION PROGRAMMING LANGUAGES

N/A. The Wi-Fi connection is a networking capability provided by the mini PC and does not require programming languages.

3.7.5 Wi-Fi CONNECTION DATA STRUCTURES

Data transmitted over the Wi-Fi connection is encapsulated into standard network packets managed by the operating system's networking stack. These packets carry application data between the kiosk and remote services. No subsystem-specific data structures are defined at this layer.

3.7.6 Wi-Fi CONNECTION DATA PROCESSING

N/A. The Wi-Fi connection is a networking capability provided by the mini PC and does not perform application-level data processing or implement algorithms.

4 FRONT-END LAYER SUBSYSTEMS

The Front-End Layer is responsible for presenting user interfaces, handling touchscreen interactions, and transmitting user input data to the back-end in support of Smart Kiosk functionality. This layer includes the Login and Registration interface, Supply Ordering interface, Interactive Map integrated with the MappedIn API, Room Reservation interface, Announcement Board, and Admin Dashboard. All front-end subsystems are implemented using React and communicate with the back-end through defined API interactions. This section provides detailed specifications for each front-end subsystem.

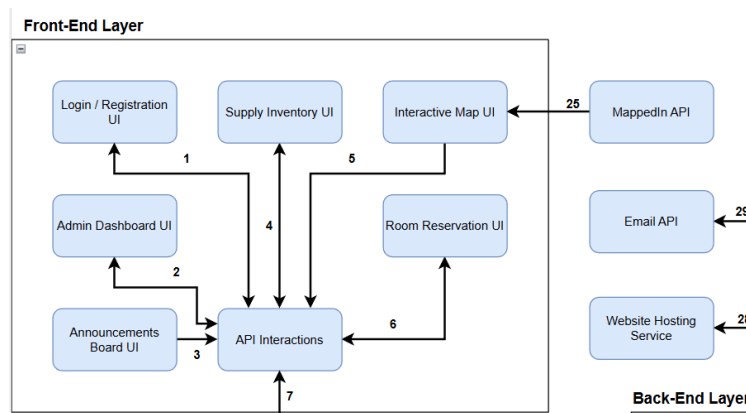


Figure 7: Front-End Layer

4.1 FRONT-END LAYER HARDWARE

The hardware supporting the Front-End Layer consists primarily of the kiosk computer and touchscreen monitor. The kiosk computer provides the computing resources required to render the React-based user interfaces, while the touchscreen monitor enables visual display and captures user interactions. No additional front-end specific hardware is required beyond the hardware defined in the Hardware Layer.

4.2 FRONT-END LAYER OPERATING SYSTEM

The Front-End Layer requires an operating system that supports the runtime environments necessary to execute a React-based application and communicate with the Django back-end. Compatible operating systems include Windows and Linux.

4.3 FRONT-END LAYER SOFTWARE DEPENDENCIES

The Front-End Layer depends on standard web technologies, including HTML, CSS, and JavaScript, to render the React-based user interface. A JavaScript runtime environment such as Node.js is required to support the front-end development and build process, while network communication enables interaction with the Django back-end through defined APIs.

4.4 SUBSYSTEM 1: LOGIN/REGISTRATION USER INTERFACE

The Login/Registration User Interface is a front-end software subsystem responsible for authenticating users and initiating user sessions within the Smart Kiosk. It is implemented as a React-based user interface rendered in the kiosk's browser environment. This subsystem allows users to log in using credentials, register new accounts, or optionally authenticate using a card swipe reader. User input is validated locally and authentication requests are transmitted to the back-end services.

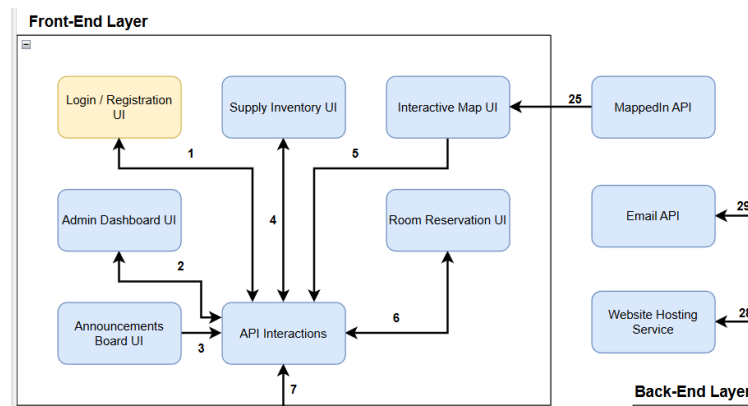


Figure 8: Login/Registration UI

4.4.1 LOGIN/REGISTRATION UI HARDWARE

The Login/Registration User Interface relies on the kiosk computer and touchscreen monitor to display interface elements and capture user input. An optional card swipe reader may be used to provide authentication data as an alternative login method.

4.4.2 LOGIN/REGISTRATION UI OPERATING SYSTEM

This subsystem operates under the same operating system requirements defined for the Front-End Layer.

4.4.3 LOGIN/REGISTRATION UI SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the Front-End Layer.

4.4.4 LOGIN/REGISTRATION UI PROGRAMMING LANGUAGES

The Login/Registration User Interface is primarily implemented using JavaScript with JSX syntax. Supporting technologies include HTML for structure and CSS for styling.

4.4.5 LOGIN/REGISTRATION UI DATA STRUCTURES

Key data structures used by this subsystem include client-side state objects representing user input and authentication state, such as email, password, card swipe data, and form validation status. Authentication requests and responses are exchanged with the back-end using JSON.

4.4.6 LOGIN/REGISTRATION UI DATA PROCESSING

The subsystem performs client-side input validation and formatting, including verifying email domains, enforcing password requirements, confirming matching credentials during registration, and handling card swipe input when available. It manages user interface state transitions based on authentication outcomes returned by the back-end, such as successful login, failed authentication, or password reset requests.

4.5 SUBSYSTEM 2: SUPPLY INVENTORY USER INTERFACE

The Supply Inventory User Interface allows users to view a list of available supply items and identify items with low stock levels. Users can select one or more items that require restocking and submit a restock request through the interface.

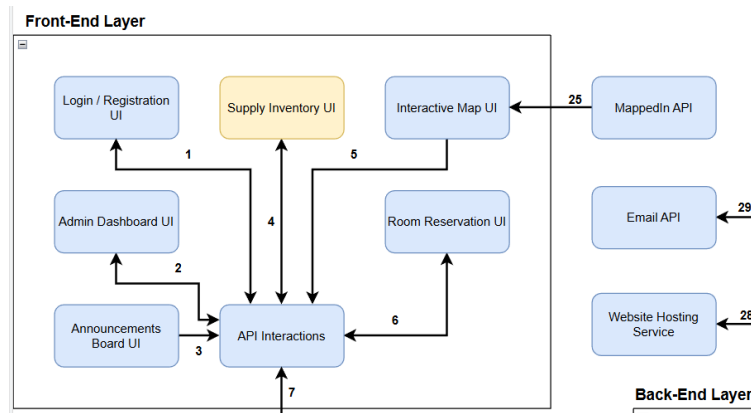


Figure 9: Supply Inventory UI

4.5.1 SUPPLY INVENTORY UI HARDWARE

The Supply Inventory User Interface relies on the kiosk computer and touchscreen monitor to display interface elements and capture user input.

4.5.2 SUPPLY INVENTORY UI OPERATING SYSTEM

This subsystem operates under the same operating system requirements defined for the Front-End Layer.

4.5.3 SUPPLY INVENTORY UI SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the Front-End Layer.

4.5.4 SUPPLY INVENTORY UI PROGRAMMING LANGUAGES

The Supply Inventory User Interface is implemented using JavaScript with JSX syntax. Supporting technologies include HTML for structure and CSS for styling.

4.5.5 SUPPLY INVENTORY UI DATA STRUCTURES

Data such as inventory item, date, and user who ordered are sent to the back-end via JSON.

4.5.6 SUPPLY INVENTORY UI DATA PROCESSING

This subsystem performs no data processing beyond UI state management. It functions as a presentational and interaction layer that forwards user selections to the back-end system for processing.

4.6 SUBSYSTEM 3: INTERACTIVE MAP USER INTERFACE

The interactive map subsystem is provided by Mappedin, a third-party mapping platform that specializes in three-dimensional indoor map rendering and optimized routing algorithms. The subsystem is integrated into the system by embedding Mappedin's web-based interface directly into the website, allowing users to interact with the map without requiring additional client-side processing or custom map logic.

4.6.1 INTERACTIVE MAP UI HARDWARE

The Interactive Map User Interface relies on the kiosk computer and touchscreen monitor to display interface elements and capture user input.

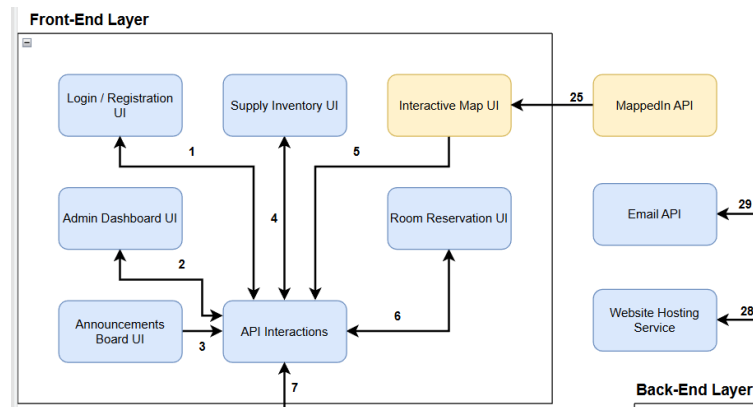


Figure 10: Interactive Map UI

4.6.2 INTERACTIVE MAP UI OPERATING SYSTEM

This subsystem requires an operating system capable of running a modern web browser that supports embedded third-party web content.

4.6.3 INTERACTIVE MAP UI SOFTWARE DEPENDENCIES

The subsystem depends on the availability of Mappedin's web-based mapping service. The embedded map is loaded through a hosted link and requires an active internet connection to display correctly.

4.6.4 INTERACTIVE MAP UI PROGRAMMING LANGUAGES

The Interactive Map subsystem is implemented using JavaScript with React and JSX syntax. The Mappedin web map is embedded within the component using an `<iframe>` element.

4.6.5 INTERACTIVE MAP UI DATA STRUCTURES

N/A. User interactions are handled directly by the embedded Mappedin interface.

4.6.6 INTERACTIVE MAP UI DATA PROCESSING

N/A. All map rendering, route computation, and interaction processing are handled exclusively by Mappedin.

4.7 SUBSYSTEM 4: ADMIN DASHBOARD USER INTERFACE

The Admin Dashboard User Interface subsystem provides authorized administrative users with tools to manage kiosk content and system data. This interface supports functions such as managing user accounts, adding or removing supply items from the inventory, modifying or canceling room reservations, and updating promotional banners or display content through a centralized, user-friendly control panel.

4.7.1 ADMIN DASHBOARD UI HARDWARE

The Admin Dashboard User Interface relies on the kiosk computer and touchscreen monitor to display interface elements and capture user input.

4.7.2 ADMIN DASHBOARD UI OPERATING SYSTEM

This subsystem operates under the same operating system requirements defined for the Front-End Layer.

4.7.3 ADMIN DASHBOARD UI SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the Front-End Layer.

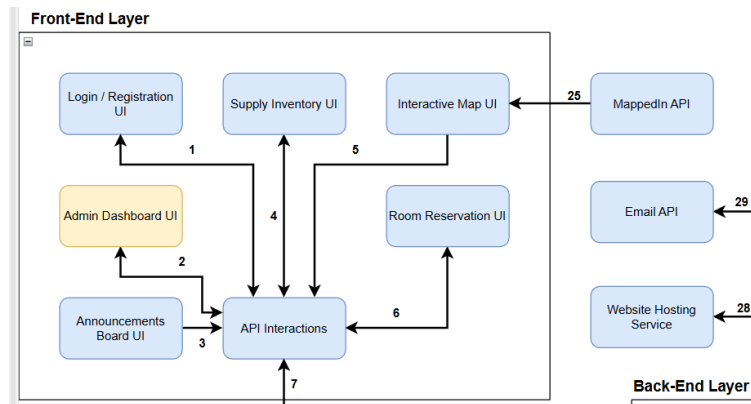


Figure 11: Admin Dashboard UI

4.7.4 ADMIN DASHBOARD UI PROGRAMMING LANGUAGES

The Admin Dashboard User Interface is primarily implemented using JavaScript with JSX syntax. Supporting technologies include HTML for structure and CSS for styling.

4.7.5 ADMIN DASHBOARD UI DATA STRUCTURES

This subsystem exchanges data with the back-end using JSON-formatted request and response objects. These data structures contain the relevant parameters required to support administrative functions described in the previous section.

4.7.6 ADMIN DASHBOARD UI DATA PROCESSING

This subsystem performs client-side input validation prior to submitting requests to the back-end. Validation includes verifying that required input fields are present and ensuring that requested actions, such as room reservation modifications, meet validity constraints before submission.

4.8 SUBSYSTEM 5: ROOM RESERVATION USER INTERFACE

The Room Reservation User Interface subsystem provides users with the ability to view available meeting rooms and their associated availability by date and time. The subsystem includes a reservation panel that allows users to input the required information to create room reservations.

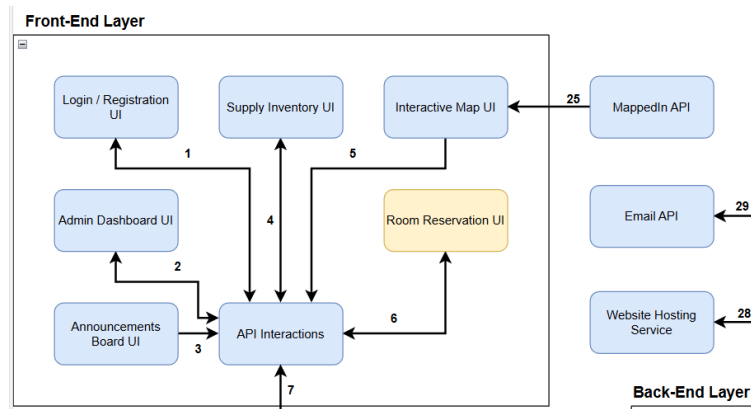


Figure 12: Room Reservation UI

4.8.1 ROOM RESERVATION UI HARDWARE

The Room Reservation User Interface relies on the kiosk computer and touchscreen monitor to display interface elements and capture user input.

4.8.2 ROOM RESERVATION UI OPERATING SYSTEM

This subsystem operates under the same operating system requirements defined for the Front-End Layer.

4.8.3 ROOM RESERVATION UI SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the Front-End Layer.

4.8.4 ROOM RESERVATION UI PROGRAMMING LANGUAGES

The Room Reservation User Interface is primarily implemented using JavaScript with JSX syntax. Supporting technologies include HTML for structure and CSS for styling.

4.8.5 ROOM RESERVATION UI DATA STRUCTURES

This subsystem exchanges room reservation data with the back-end using JSON-formatted request objects.

4.8.6 ROOM RESERVATION UI DATA PROCESSING

This subsystem performs client-side validity checks prior to submitting reservation requests to the back-end. These checks include identifying obvious scheduling conflicts and verifying that required reservation fields are completed.

4.9 SUBSYSTEM 6: ANNOUNCEMENTS BOARD USER INTERFACE

The Announcements Board User Interface subsystem displays informational banners and panels for kiosk users, including promotional, holiday, and event-specific announcements.

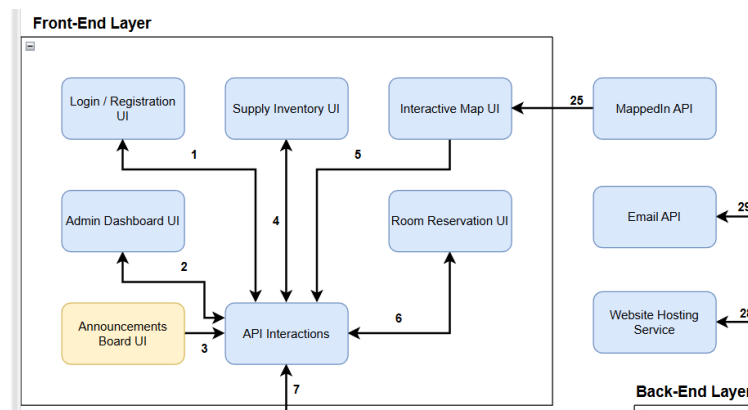


Figure 13: Announcement Board UI

4.9.1 ANNOUNCEMENTS BOARD UI HARDWARE

The Announcement Board User Interface relies on the kiosk computer and touchscreen monitor to display interface elements.

4.9.2 ANNOUNCEMENTS BOARD UI OPERATING SYSTEM

This subsystem operates under the same operating system requirements defined for the Front-End Layer.

4.9.3 ANNOUNCEMENTS BOARD UI SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the Front-End Layer.

4.9.4 ANNOUNCEMENTS BOARD UI PROGRAMMING LANGUAGES

The Announcement Board User Interface is primarily implemented using JavaScript with JSX syntax. Supporting technologies include HTML for structure and CSS for styling.

4.9.5 ANNOUNCEMENTS BOARD UI DATA STRUCTURES

This subsystem receives JSON-formatted data from the back-end that specifies the announcement content to be displayed.

4.9.6 ANNOUNCEMENTS BOARD UI DATA PROCESSING

N/A. This subsystem does not perform data processing and is limited to rendering the provided announcement content.

4.10 SUBSYSTEM 7: API INTERACTIONS

The API Interactions subsystem consists of a React-based module responsible for managing communication between the front-end user interface subsystems and the Django back-end. This subsystem centralizes outgoing JSON-formatted API requests generated by the previously defined subsystems and handles incoming JSON responses from the back-end.

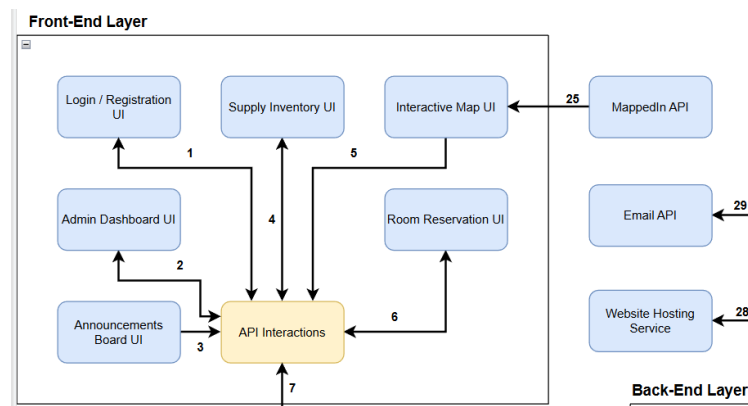


Figure 14: API Interactions

4.10.1 API INTERACTIONS HARDWARE

The API Interaction subsystem relies on the kiosk computer to operate.

4.10.2 API INTERACTIONS OPERATING SYSTEM

This subsystem operates under the same operating system requirements defined for the Front-End Layer.

4.10.3 API INTERACTIONS SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the Front-End Layer.

4.10.4 API INTERACTIONS PROGRAMMING LANGUAGES

The API Interaction subsystem is primarily implemented using JavaScript with JSX syntax.

4.10.5 API INTERACTIONS DATA STRUCTURES

This subsystem exchanges data with the back-end using JSON-formatted request and response objects. These data structures encapsulate parameters and payloads generated by the front-end user interface subsystems and provide a standardized format for API communication.

4.10.6 API INTERACTIONS DATA PROCESSING

N/A. This subsystem performs no data processing.

5 BACK-END LAYER SUBSYSTEMS

The back end layer is responsible for managing all the server-side logic for the application and communicating updates effectively with the Front-End and Database layers of the application. It is integrated with the other components to process and serve the users' requests to create a smooth and productive user experience. This layer includes the Django Python Framework, the Login Manager, Supply Order Manager, Automated Email System, Room Reservation Manager, Announcement Manager, Admin's Portal Manager, and Website Hosting Service. All back end subsystems are implemented using Django and communicate with the front-end through defined API interactions. This section provides detailed specifications for each back end subsystem.

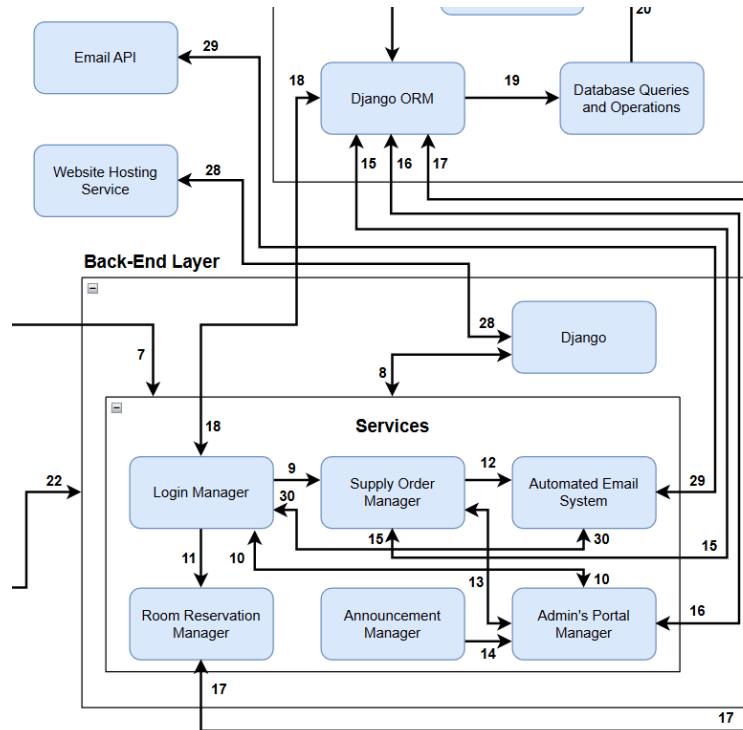


Figure 15: Back-End Layer

5.1 BACK-END LAYER HARDWARE

The hardware supporting the back end Layer consists primarily of the kiosk computer and touchscreen monitor. The kiosk computer provides the computing resources required to handle the Django logic and communication, while the touchscreen monitor enables visual display and captures user interactions. No additional back end specific hardware is required beyond the hardware defined in the Hardware Layer.

5.2 BACK-END LAYER OPERATING SYSTEM

The back end Layer requires an operating system that supports the runtime environments necessary to execute a React-based application and communicate with the Django back end. Compatible operating systems include Windows and Linux.

5.3 BACK-END LAYER SOFTWARE DEPENDENCIES

The back end Layer utilizes Python 3 as the structure and foundation for handling all logic and communication, along with REST application programming interfaces. The libraries and frameworks required to run the back end are found in a requirements text file, which can be installed using the Python Package Installer (PIP). These include Django 5.0 and its related libraries such as crispy-forms 2.1, bootstrap 2024.1, and the Django REST framework.

5.4 SUBSYSTEM 1: DJANGO

Django is a high-level Python web framework used to quickly, cleanly, and safely build web applications. [1] It has a large community and user base thanks to it being free to use and open-source. Django handles user authentication, administration, and many more common web application tasks and works smoothly with other frameworks.

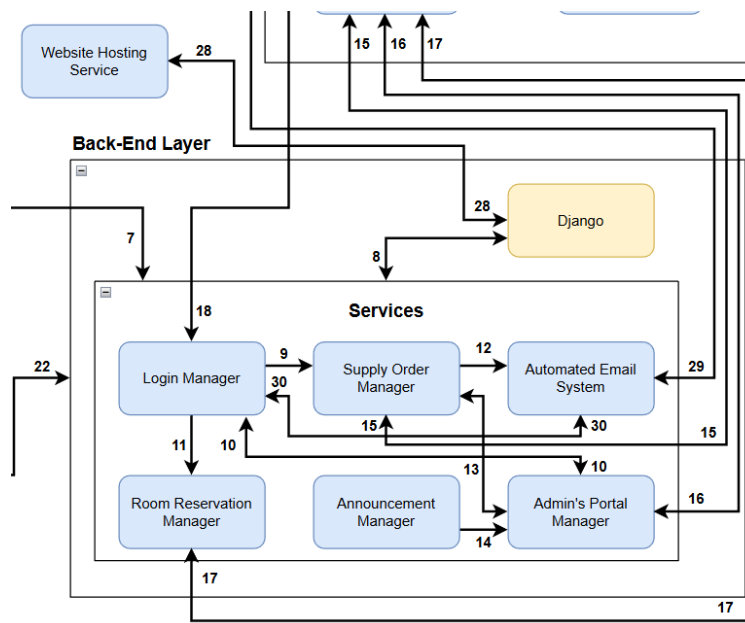


Figure 16: Django

5.4.1 DJANGO HARDWARE

N/A

5.4.2 DJANGO OPERATING SYSTEM

This subsystem utilizes the same operating system as the overall layer. [Windows or Linux]

5.4.3 DJANGO SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the back end Layer.

5.4.4 DJANGO PROGRAMMING LANGUAGES

This subsystem utilizes Python 3.0 and up.

5.4.5 DJANGO DATA STRUCTURES

Creates database tables provided the relevant schema using Django ORM.

5.4.6 DJANGO DATA PROCESSING

Django utilizes its Django Object Relational Mapping (ORM) to handle data and Serialization to implement data validation. It communicates with the Database Layer to perform this processing.

5.5 SUBSYSTEM 2: LOGIN MANAGER

This subsystem is a feature implemented through Django and REST APIs. The Login Manager is in charge of handling all the logic involved with signing in and signing up user accounts, password management, and logging in to the Smart Kiosk through the swipe-card functionality. The Login Manager adds security through verification, validation, and authentication. It communicates with the Database Layer to function effectively.

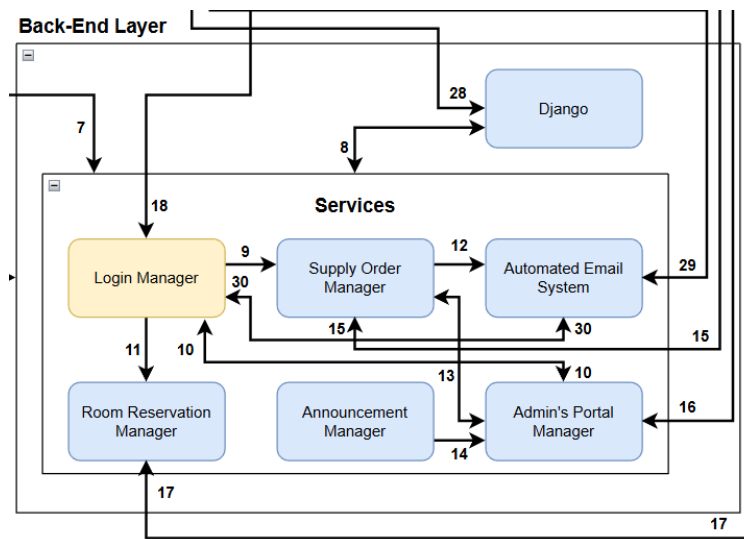


Figure 17: Login Manager

5.5.1 LOGIN MANAGER HARDWARE

The hardware for this subsystem uses the same hardware as the overall layer.

5.5.2 LOGIN MANAGER OPERATING SYSTEM

This subsystem utilizes the same operating system as the overall layer.

5.5.3 LOGIN MANAGER SOFTWARE DEPENDENCIES

The Login Manager operates under the same software dependencies requirements defined for the back end Layer.

5.5.4 LOGIN MANAGER PROGRAMMING LANGUAGES

The Login Manager is implemented using Python through the Django web framework.

5.5.5 LOGIN MANAGER DATA STRUCTURES

The Login Manager uses authentication classes that perform data validation methods along with hash maps to store HTTP headers and user credentials.

5.5.6 LOGIN MANAGER DATA PROCESSING

The Login Manager utilizes Django's ORM to retrieve and validate user credentials from the database. Django serializers are used for input validation. Additionally, authentication logic is processed before

returning a structured JSON response to the front-end. For password resets, Amazon SES is utilized to send the user a temporary code.

5.6 SUBSYSTEM 3: SUPPLY ORDER MANAGER

This subsystem is a back end component implemented through Django and Automated Email sending services. It communicates with the hardware and database layer to operate efficiently. The Supply Order Manager is responsible for processing the logic behind supply order requests captured through the user interface. The subsystem retrieves the relevant information from the database, constructs an automated email containing the selected items, and sends it to the Supply Manager using an email service.

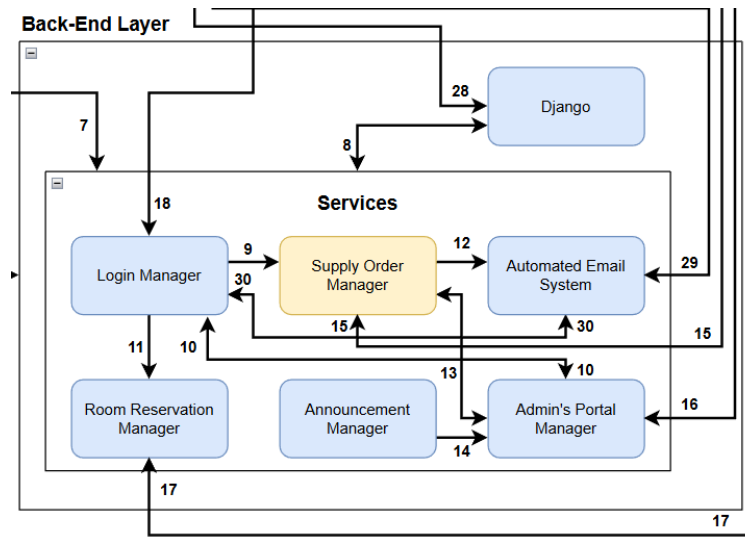


Figure 18: Supply Order Manager

5.6.1 SUPPLY ORDER MANAGER HARDWARE

The hardware for this subsystem uses the same hardware as the overall layer.

5.6.2 SUPPLY ORDER MANAGER OPERATING SYSTEM

This subsystem utilizes the same operating system as the overall layer.

5.6.3 SUPPLY ORDER MANAGER SOFTWARE DEPENDENCIES

The Supply Order Manager operates under the same software dependencies requirements defined for the back end Layer.

5.6.4 SUPPLY ORDER MANAGER PROGRAMMING LANGUAGES

The Supply Order Manager is implemented using Python through the Django web framework.

5.6.5 SUPPLY ORDER MANAGER DATA STRUCTURES

The Supply Order Manager stores data through JSON objects.

5.6.6 SUPPLY ORDER MANAGER DATA PROCESSING

The Supply Order Manager utilizes Django's ORM in conjunction with Amazon SES to send the designated supply manager an email with the user's supply request.

5.7 SUBSYSTEM 4: AUTOMATED EMAIL SYSTEM

This subsystem is a back end component implemented through Django and AWS Simple Email Service (SES). It communicates with the hardware and database layer to operate efficiently. The Automated Email System is responsible for processing the logic that triggers emails to be automatically sent to the user/administrator. These emails consist of information on updates regarding user accounts, updates to users' room reservations, and supply request notifications. The Automated Email System allows users to keep up-to-date with the services provided by the Kiosk.

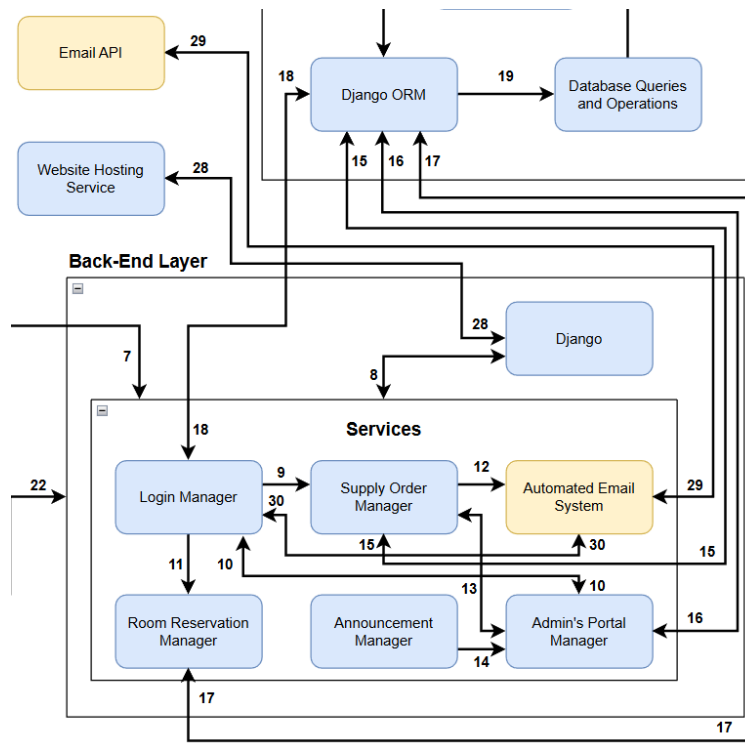


Figure 19: Automated Email System with API

5.7.1 AUTOMATED EMAIL SYSTEM HARDWARE

The hardware for this subsystem uses the same hardware as the overall layer, and the computing power of AWS.

5.7.2 AUTOMATED EMAIL SYSTEM OPERATING SYSTEM

This subsystem utilizes the same operating system as the overall layer.

5.7.3 AUTOMATED EMAIL SYSTEM SOFTWARE DEPENDENCIES

This subsystem primarily utilizes SMTP and Python AWS SDK's. Additionally, the Automated Email System operates under the same software dependencies and requirements defined for the back end Layer.

5.7.4 AUTOMATED EMAIL SYSTEM PROGRAMMING LANGUAGES

The Automated Email System is implemented through Python.

5.7.5 AUTOMATED EMAIL SYSTEM DATA STRUCTURES

The Automated Email System utilizes Django Model objects, and email templates are formed using HTML strings

5.7.6 AUTOMATED EMAIL SYSTEM DATA PROCESSING

Data is processed through user-driven events and is processed by AWS Simple Email Service (SES).

5.8 SUBSYSTEM 5: ROOM RESERVATION MANAGER

This subsystem is a back end component implemented through Django and is integrated with Microsoft Outlook. It communicates with the front-end and database layers to operate efficiently. The Room Reservation Manager is responsible for processing the logic behind any room reservation updates made by the user. This consists of displaying room statuses and current reservations, creating new reservations, and updating and deleting reservations.

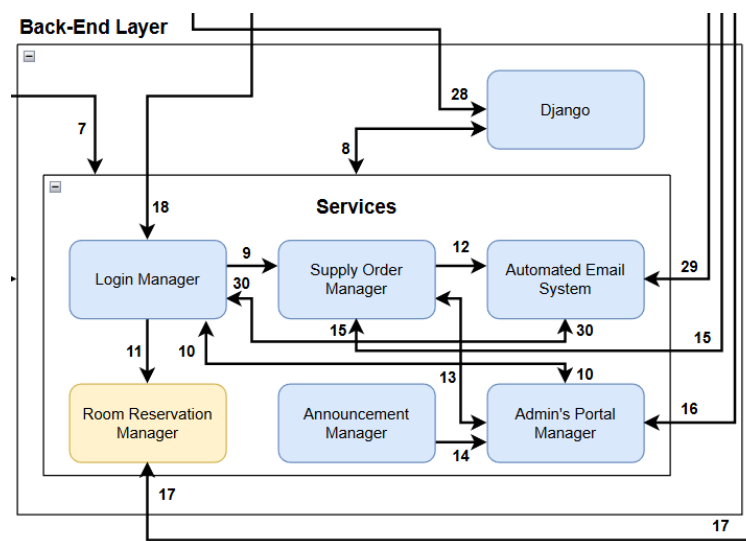


Figure 20: Room Reservation Manager

5.8.1 ROOM RESERVATION MANAGER HARDWARE

The hardware for this subsystem uses the same hardware as the overall layer.

5.8.2 ROOM RESERVATION MANAGER OPERATING SYSTEM

This subsystem utilizes the same operating system as the overall layer.

5.8.3 ROOM RESERVATION MANAGER SOFTWARE DEPENDENCIES

This subsystem primarily utilizes the latest version of Outlook. Additionally, the Room Reservation Manager operates under the same software dependencies and requirements defined for the back end Layer.

5.8.4 ROOM RESERVATION MANAGER PROGRAMMING LANGUAGES

The Room Reservation Manager is implemented through Python.

5.8.5 ROOM RESERVATION MANAGER DATA STRUCTURES

Room Reservation data is formatted through JSON-formatted request objects.

5.8.6 ROOM RESERVATION MANAGER DATA PROCESSING

The Room Reservation Manager utilizes Django's ORM to retrieve current reservations and set new reservations. Users will then receive an email with their reservation confirmation details. Amazon SES handles email processing.

5.9 SUBSYSTEM 6: ANNOUNCEMENT MANAGER

This subsystem is a back end component implemented through Django. It communicates with the front-end and database layers to operate efficiently. The Announcement Manager is responsible for processing the logic behind displaying the banners on the Smart Kiosk, including how long each banner is displayed before rotating.

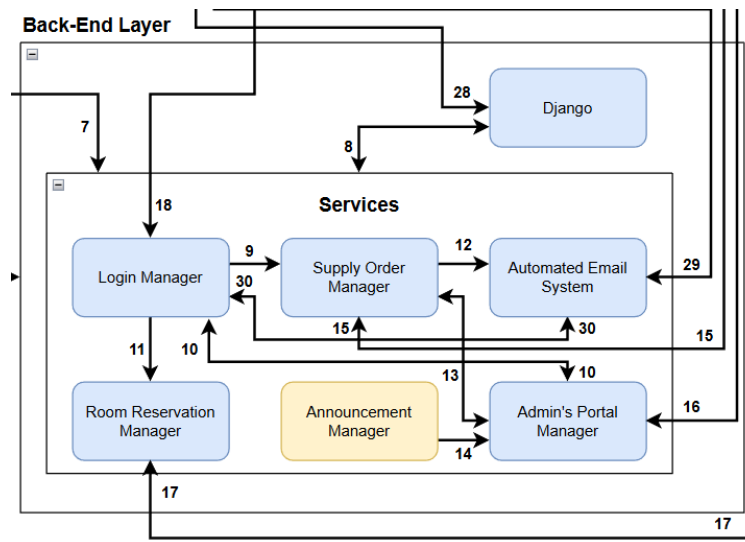


Figure 21: Announcement Manager

5.9.1 ANNOUNCEMENT MANAGER HARDWARE

The hardware for this subsystem uses the same hardware as the overall layer.

5.9.2 ANNOUNCEMENT MANAGER OPERATING SYSTEM

This subsystem utilizes the same operating system as the overall layer.

5.9.3 ANNOUNCEMENT MANAGER SOFTWARE DEPENDENCIES

The Announcement Manager operates under the same software dependencies requirements defined for the back end Layer

5.9.4 ANNOUNCEMENT MANAGER PROGRAMMING LANGUAGES

The Announcement Manager is implemented using Python

5.9.5 ANNOUNCEMENT MANAGER DATA STRUCTURES

The Announcement Manager displays JPG and PNG images through JSON response objects, depending on the banner schedule set by the admin.

5.9.6 ANNOUNCEMENT MANAGER DATA PROCESSING

The Announcement Manager utilizes Django's ORM to retrieve the activated banner images. It communicates with the Admin Portal Manager by allowing admins to set the active banner images.

5.10 SUBSYSTEM 7: ADMIN'S PORTAL MANAGER

The Admin's Portal Manager is responsible for processing the logic used for administrative functionalities such as managing user accounts, updating supplies, updating rooms and reservations, and the decorative banners. This subsystem is only available to accounts that are registered as administrators. This subsystem communicates with the database layer to operate efficiently. This subsystem is a back end component implemented through Django.

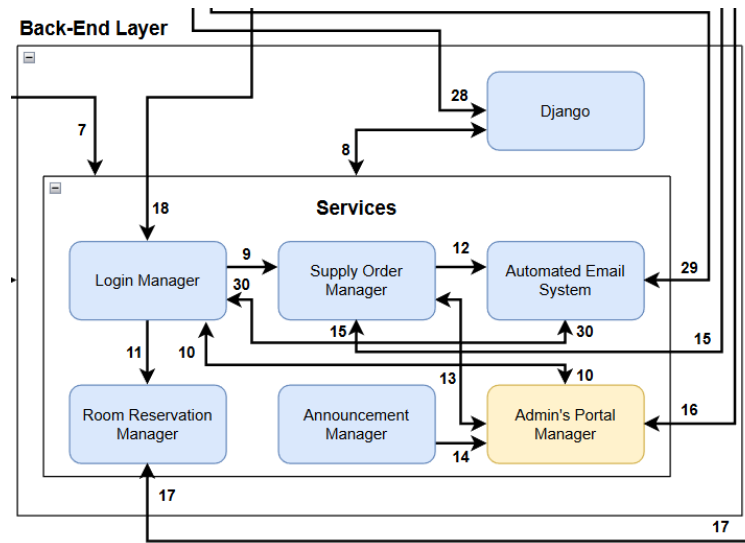


Figure 22: Admin's Portal Manager

5.10.1 ADMIN'S PORTAL MANAGER HARDWARE

The hardware for this subsystem uses the same hardware as the overall layer.

5.10.2 ADMIN'S PORTAL MANAGER OPERATING SYSTEM

This subsystem utilizes the same operating system as the overall layer.

5.10.3 ADMIN'S PORTAL MANAGER SOFTWARE DEPENDENCIES

The Announcement Manager operates under the same software dependencies requirements defined for the back end Layer

5.10.4 ADMIN'S PORTAL MANAGER PROGRAMMING LANGUAGES

The Admin's Portal Manager is implemented using Python

5.10.5 ADMIN'S PORTAL MANAGER DATA STRUCTURES

The Admin's Portal Manager utilizes the same structured Django model objects across the Login, Supply Order, Room Reservation, and Announcement subsystem managers.

5.10.6 ADMIN'S PORTAL MANAGER DATA PROCESSING

It leverages Django serializers for input validation and structured JSON request objects to perform create, update, and delete operations on existing records. These modifications are persisted to the database through Django's ORM.

5.11 SUBSYSTEM 8: WEBSITE HOSTING SERVICE

The back end layer features a subsystem that uses a website hosting service called Render. Render is a Platform as a Service (PaaS) that allows users to deploy and scale web applications through its 'Web Service' service type. [2] Render works with the Git repository on GitHub to build the code written using the Django framework, along with the ReactJS Front-end. Once completed, it returns a 'onrender.com' subdomain to allow the application to receive traffic over HTTP.

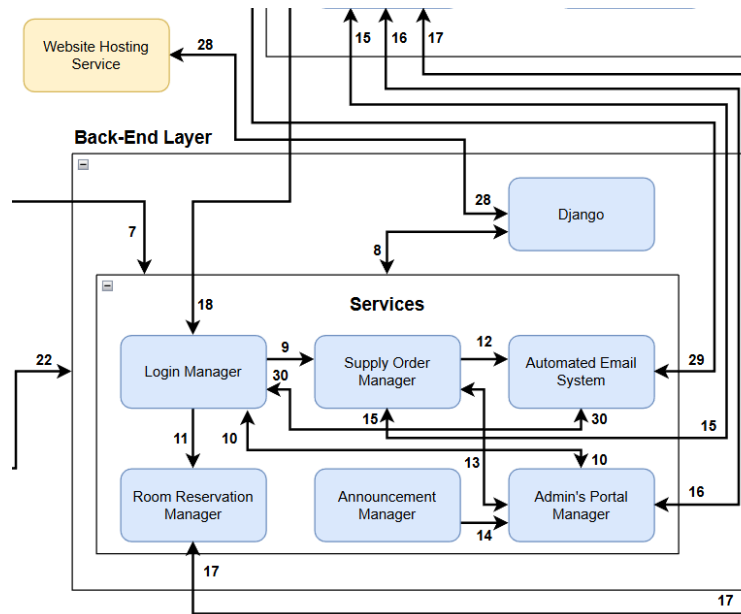


Figure 23: Website Hosting Service

5.11.1 WEBSITE HOSTING SERVICE HARDWARE

The hardware for this subsystem uses the same hardware as the overall layer, along with Render's combination of GCP and AWS instances

5.11.2 WEBSITE HOSTING SERVICE OPERATING SYSTEM

This subsystem utilizes the same operating system as the overall layer.

5.11.3 WEBSITE HOSTING SERVICE SOFTWARE DEPENDENCIES

The Website Hosting Service utilizes the latest version of Git

5.11.4 WEBSITE HOSTING SERVICE PROGRAMMING LANGUAGES

N/A

5.11.5 WEBSITE HOSTING SERVICE DATA STRUCTURES

The Website Hosting Service uses environment variables through hash-maps to securely configure data during deployment

5.11.6 WEBSITE HOSTING SERVICE DATA PROCESSING

The Website Hosting Service processes deployment workflows by pulling source code from the Git repositories in order to install dependencies, build the application environment, and route incoming HTTP requests to the appropriate handlers.

6 DATABASE LAYER SUBSYSTEMS

The Database Layer of the Smart Kiosk is responsible for storing, managing, and retrieving all data required by the Back End Layer. It supports system functionalities by handling data such as user credentials, supply inventory records, room reservation information, display content, and promotional banners. This layer is composed of the TiDB Cloud Database, the Django ORM subsystem, and the Database Queries and Operations subsystem, which work together to provide secure, scalable, and efficient data management. The following sections present detailed specifications for each of these subsystems.

6.1 DATABASE LAYER HARDWARE

The Database Layer requires a computer capable of running the Django back end and supporting secure communication with the TiDB Cloud database service. The hardware must provide sufficient processing power, memory, and network connectivity to handle database queries and management operations in real time.

6.2 DATABASE LAYER OPERATING SYSTEM

The Database Layer requires an operating system that supports the Python runtime environment and Django framework in order to communicate with the TiDB Cloud database service. Compatible operating systems include Microsoft Windows and Linux distributions. The operating system must also support secure network communication protocols required for encrypted database connections.

6.3 DATABASE LAYER SOFTWARE DEPENDENCIES

The Database Layer depends on the Django framework specifically for its Object Relational Mapping functionality, which enables structured interaction with the database. Communication with the TiDB Cloud database is handled through the PyMySQL driver. This allows Django to connect using the MySQL-compatible protocol supported by TiDB.

6.4 SUBSYSTEM 1: TiDB

The TiDB subsystem is an externally hosted cloud database service that provides storage and real-time data management for the Smart Kiosk system. It stores user credentials, room reservations, supply inventory records, and display content used throughout the application.

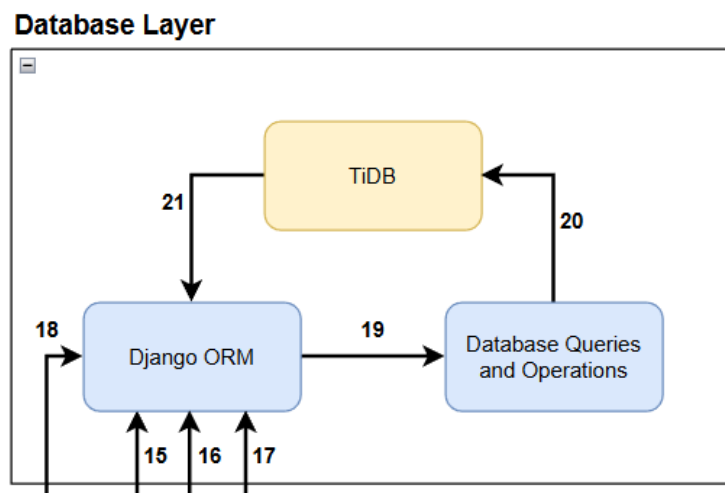


Figure 24: TiDB

6.4.1 TiDB HARDWARE

TiDB operates as a Software as a Service cloud platform and therefore relies on managed cloud infrastructure provided by the hosting vendor. The Smart Kiosk system does not require direct access to or management of the underlying physical hardware.

6.4.2 TiDB OPERATING SYSTEM

This subsystem operates under the same operating system requirements defined for the Database Layer.

6.4.3 TiDB SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the Database Layer.

6.4.4 TiDB PROGRAMMING LANGUAGES

TiDB supports SQL using the MySQL protocol. The Smart Kiosk back end communicates with TiDB using Python through the Django framework and Object Relational Mapping interface.

6.4.5 TiDB DATA STRUCTURES

The primary data structures utilized by this subsystem are relational database tables composed of rows and columns. These tables represent entities such as users, reservations, inventory records, and announcements. Relationships between tables are maintained using primary keys and foreign key constraints as defined in the Django models.

6.4.6 TiDB DATA PROCESSING

TiDB processes incoming SQL queries generated by the Django Object Relational Mapping framework. These operations include creating, reading, updating, and deleting transactions, lookups of credentials, reservation validations, and inventory updates.

6.5 SUBSYSTEM 2: DJANGO ORM

The Django Object Relational Mapping subsystem is a software component within the Django framework that provides communication between the application and the relational database. It is a library level subsystem that enables developers to define database schemas using Python classes called models [3]. The ORM converts these model definitions and query operations into Structured Query Language (SQL) statements that are executed by the TiDB database.

6.5.1 DJANGO ORM HARDWARE

This subsystem operates under the same hardware requirements defined for the Database Layer.

6.5.2 DJANGO ORM OPERATING SYSTEM

This subsystem operates under the same operating system requirements defined for the Database Layer.

6.5.3 DJANGO ORM SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the Database Layer.

6.5.4 DJANGO ORM PROGRAMMING LANGUAGES

This subsystem operates under the same programming language requirements defined for the TiDB subsystem.

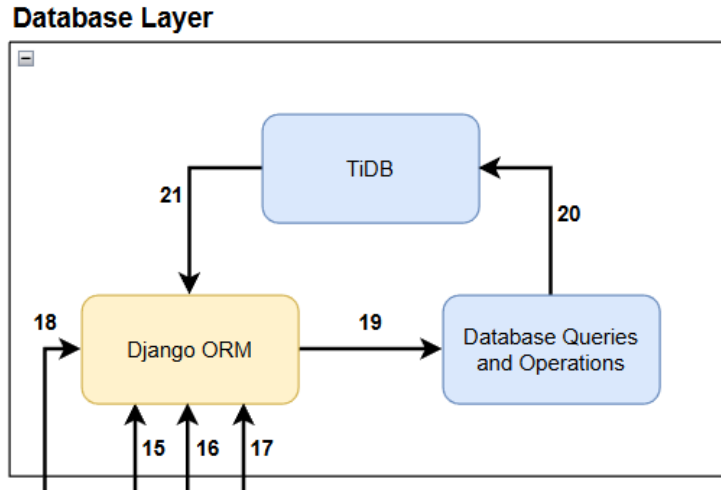


Figure 25: Django ORM

6.5.5 DJANGO ORM DATA STRUCTURES

The primary data structures used by this subsystem are Python classes referred to as models. Each model represents a relational database table and contains fields that correspond to table columns. Relationships between tables are represented using foreign key, one to one, and many to many field definitions. These structures enable object based manipulation of relational data.

6.5.6 DJANGO ORM DATA PROCESSING

The Django ORM processes data by translating Python based model operations into Structured Query Language statements that are executed by the TiDB database. After receiving query results from TiDB, the ORM converts database rows into Python objects which can then be serialized into JavaScript Object Notation format for use in the application programming interface layer.

6.6 SUBSYSTEM 3: DATABASE QUERIES AND OPERATIONS

The Database Queries and Operations subsystem is a software level component within the Django back end responsible for defining and executing application specific database interactions. It defines how data is retrieved, validated, filtered, modified, and stored in response to user actions such as login attempts, room reservations, supply requests, and content updates. This subsystem is implemented as Python methods and class based views that invoke ORM operations to perform structured database transactions.

6.6.1 DATABASE QUERIES AND OPERATIONS HARDWARE

This subsystem operates under the same hardware requirements defined for the Database Layer.

6.6.2 DATABASE QUERIES AND OPERATIONS OPERATING SYSTEM

This subsystem operates under the same operating system requirements defined for the Database Layer.

6.6.3 DATABASE QUERIES AND OPERATIONS SOFTWARE DEPENDENCIES

This subsystem operates under the same software dependencies requirements defined for the Database Layer.

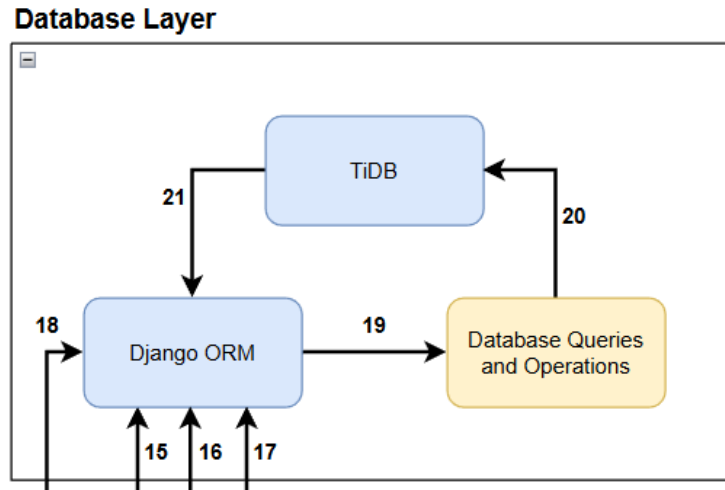


Figure 26: Database Queries and Operations

6.6.4 DATABASE QUERIES AND OPERATIONS PROGRAMMING LANGUAGES

This subsystem operates under the same programming language requirements defined for the TiDB subsystem.

6.6.5 DATABASE QUERIES AND OPERATIONS DATA STRUCTURES

The primary data structure utilized by this subsystem is the Django model instances object. Model instances represent individual records retrieved from or written to the database.

6.6.6 DATABASE QUERIES AND OPERATIONS DATA PROCESSING

This subsystem processes data by executing create, read, update, and delete operations based on user requests and rules defined in the application logic. It is utilized in validation checks such as verifying unique email addresses during registration, detecting scheduling conflicts for room reservations, and ensuring inventory availability before approving supply requests. The subsystem also uses filtering and aggregation operations provided by the ORM to efficiently retrieve structured data.

7 APPENDIX A

REFERENCES

- [1] Django Software Foundation, “Django overview,” <https://www.djangoproject.com/start/overview/>, n.d., accessed: 2026-02-12.
- [2] Render, “Web services,” <https://render.com/docs/web-services>, n.d., accessed: 2026-02-12.
- [3] Django Software Foundation, “Django orm documentation,” <https://docs.djangoproject.com/en/6.0/topics/db/queries/>, n.d., accessed: 2026-02-12.